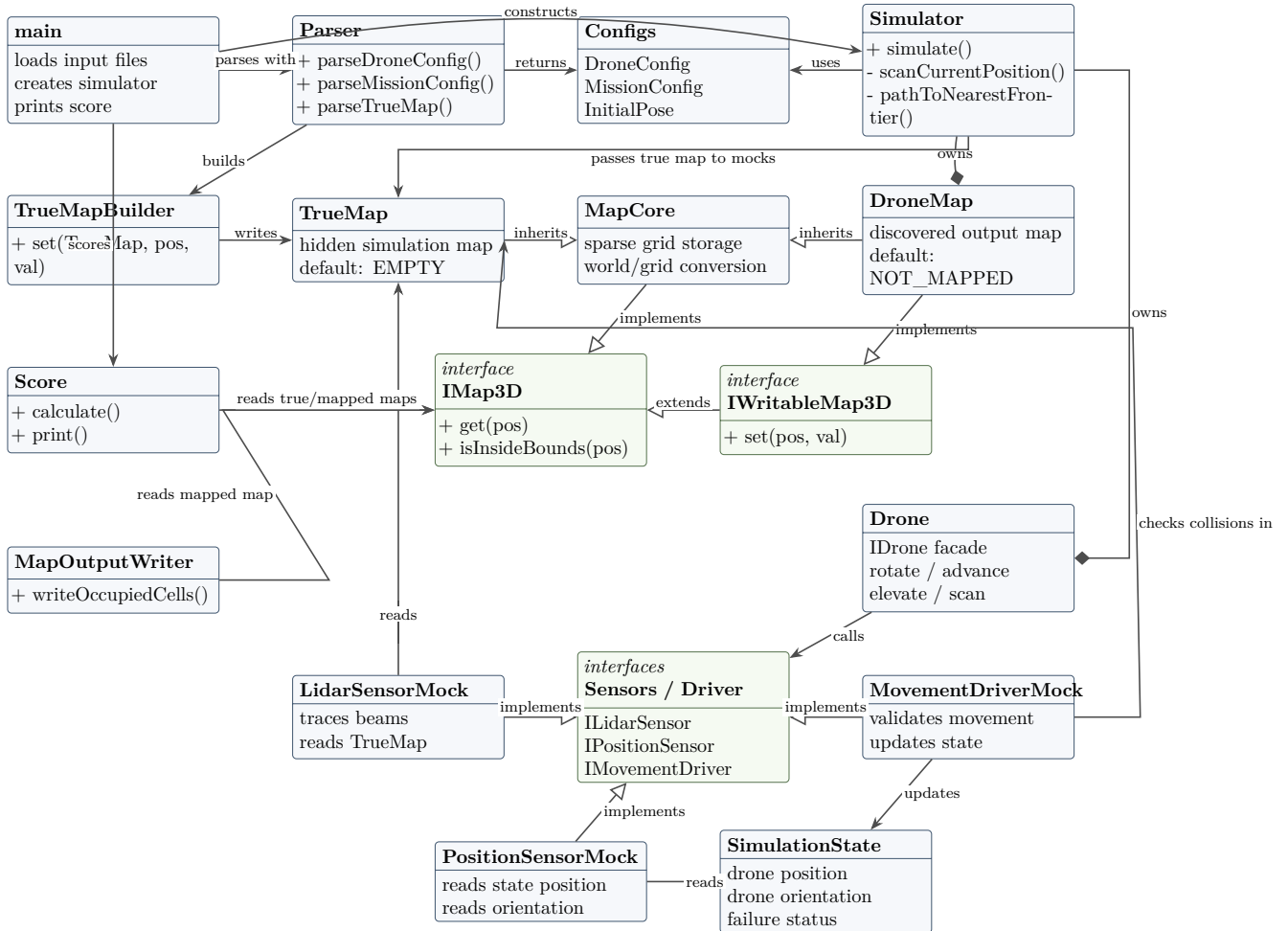


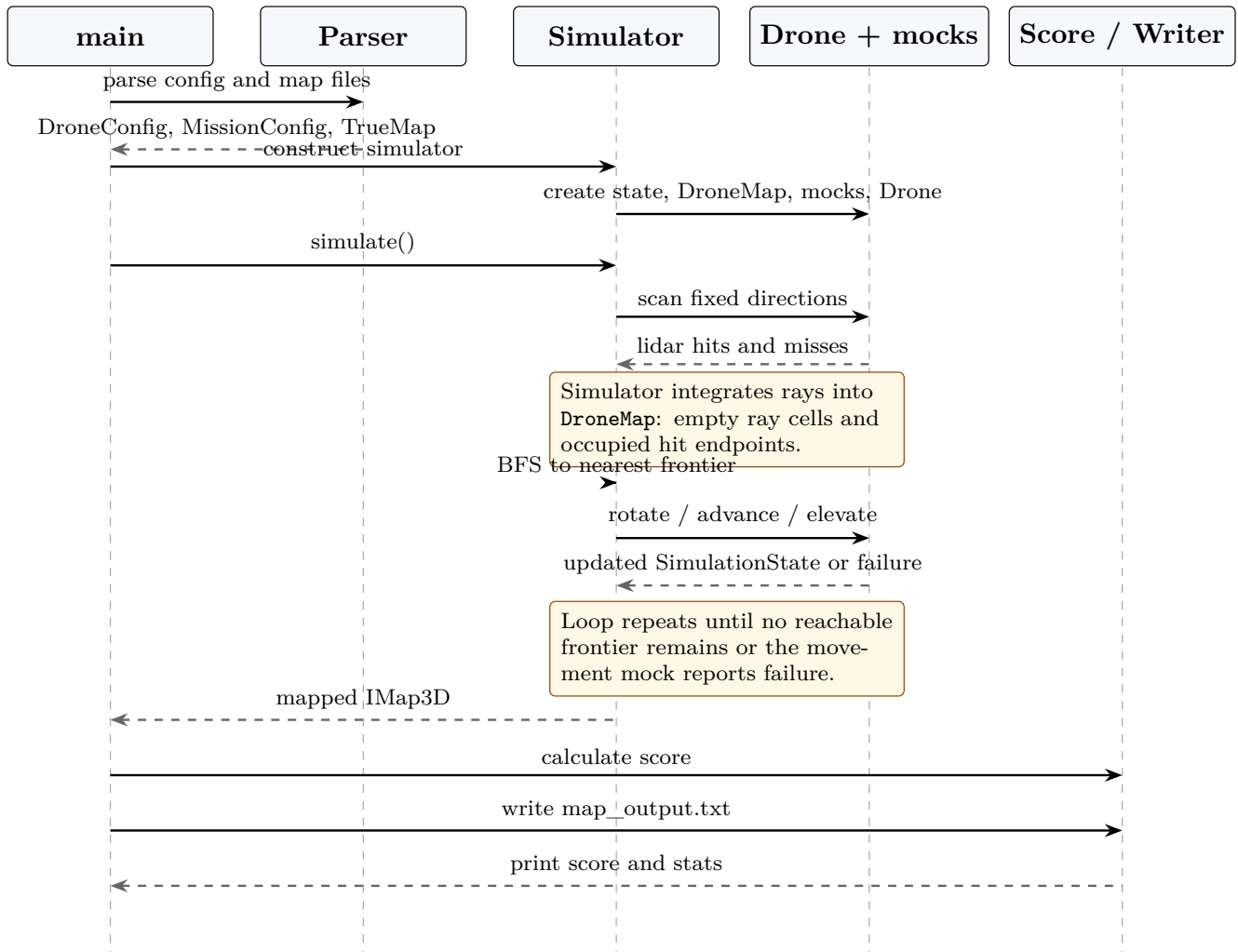
Drone Lidar

High Level Design

1 UML Class Diagram



2 UML Sequence Diagram: Main Flow



3 Design Considerations

The main thing that we thought and designed around was how to make the drone not be aware that he is using mocks and that he is inside a simulation, we achieved that using the abstract classes `ILidarSensor`, `IPositionSensor`, and `IMovementDriver`. Then the `Drone` does not know that he is using mocks, for him he is just using an existing interface that works.

4 Alternatives Considered

One of the things we considered was where should the mapping algorithm be implemented, inside the `Drone` or the `Simulator`.

We later on decided that it will be implemented inside the `Simulator`, that way the algorithm is not coupled to the `Drone` and in the future we will be able to test many different algorithms inside the `Simulator` and compare results between them.

5 Testing Approach

The automated tests use GTest and focus on the main components independently:

- Parser tests validate config parsing, map parsing, missing files, malformed lines, unknown keys, boundary checks, aliases, and the optional initial drone pose.
- Map tests validate `TrueMap`, `DroneMap`, sparse storage behavior, and boundary handling.
- Lidar tests validate scan and beam behavior against known map layouts.
- Movement driver tests validate advancing, rotating, elevation, command clamping, collision detection, boundary detection, and clearance checks.
- Simulator tests validate that simple occupied cells and free ray cells are mapped, and that the simulator starts from the mission initial position when one is configured.

The repository also includes complete scenario folders with input files and saved `original_output/map_output.txt` files. These scenarios are intended for manual runs and grader execution of complete program flows.